

Reference counting

שיטה זו נועדה מאפשרת מצב בו מספר אובייקטים עם אותו ערך, יחלקו ביניהם את אותו ייצוג (יחיד) של הערך. ישנם שני יתרונות עיקריים לשיטה:

✚ כדי לשחרר זיכרון עבור אובייקט, שהוקצה דינאמית, יש לדעת מי הבעלים שלו. רק לבעלים מותר לשחרר אותו. ייתכן וגם הוחלפו בעלים אגב ריצת התכנית. ספירת ייחוסים (Reference counting) פותרת את בעיית הבעלות: האובייקט הוא הבעלים של עצמו, למרות הרבים יכולים להשתמש בו. הוא יודע כמה משתמשים בו, וכשמספרם יורד עד ל-0 הוא דואג להרוס את עצמו.

✚ אם להרבה אובייקטים יש את אותו ערך, זו כפילות מיותרת. יכולת לחלוק את אותו ערך תחסוך מקום בזיכרון וגם זמן: פחות בנייה והריסה של אובייקטים!

מתי למשל קורה שיש כפילויות?
למשל כך:

```
class String
{
public:

    String(const char *value = "");
    String& operator=(const String& rhs);
    ...

private:
    char *data;
};

String a, b, c, d, e;

a = b = c = d = e = "Hello";
```

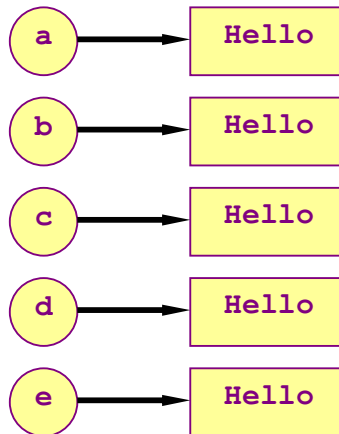
בדרך כלל כל אובייקט String נושא עימו את המחזרות שלו.
פונקציית ההשמה תיראה פחות או יותר כך:

```
String& String::operator=(const String& rhs)
{
    if (this == &rhs) return *this;

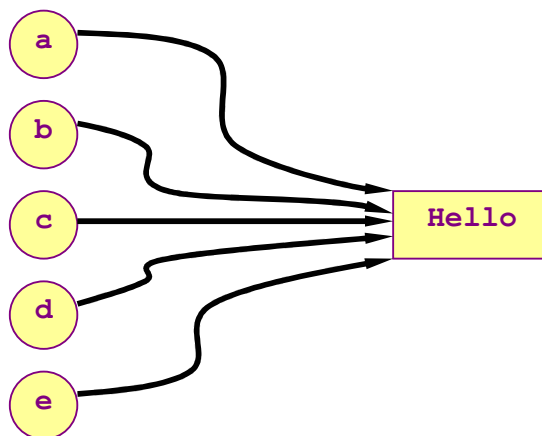
    delete [] data;
    data = new char[strlen(rhs.data) + 1];
    strcpy(data, rhs.data);

    return *this;
}
```

ולוגית נקבל משהו כזה:



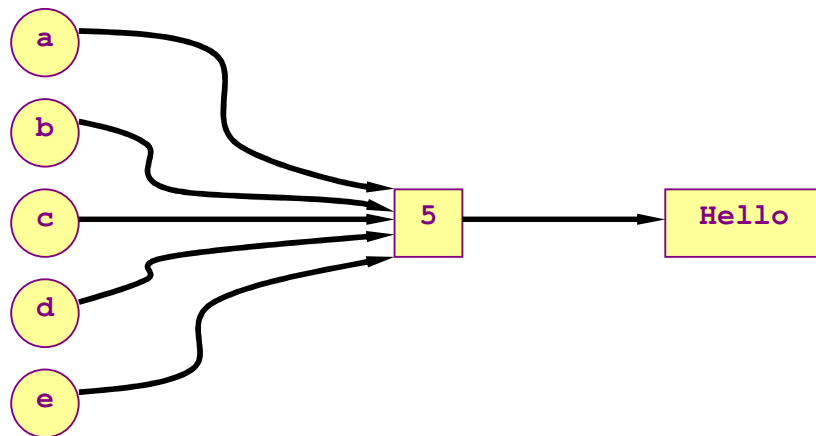
היינו רוצים לקבל מצב כזה:



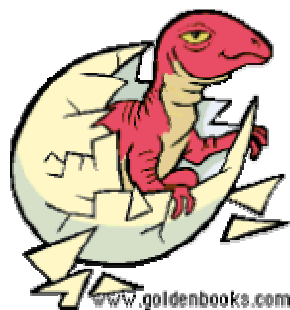
המצב האידיאלי: כל האובייקטים שרוצים באותו ערך, חולקים אותו.

זה לא כל כך פשוט: צריך לזכור כמה אובייקטים חולקים כעת את אותו ערך. בעיות:
+ אם אובייקט a מקבל ערך אחר, לא ניתן לשנות את הערך באופן רגיל, כי אז נשנה אותו עבור כל האובייקטים.
+ אם רק אובייקט אחד משתמש בערך hello, והוא כרגע נהרס, אף אחד כבר לא מתייחס לערך, וצריך להשמידו.

צריך להוסיף איזשהו מימוש ליכולת לעקוב אחרי עניין המספור.



את זה צריך לממש.
איך?



מימוש של מספור ייחוסים (Reference Counting)

צעד ראשון הוא לדעת איפה לאחסן את השדה של המספור עבור כל ערך, אליו יכולים האובייקטים להתייחס. מה יקרה אם הוא יהיה בתוך String?
אז נקבל מספור עבור כל אובייקט.
אבל אנו רוצים מספור עבור כל ערך.
אי לכך יש ליצור מחלקה מיוחדת עבור:
הערך אליו ניתן להתייחס. 
המספור עבור אותו ערך – כמה מתייחסים אליו כרגע. 

היות וזו מחלקת עזר עבור String בלבד, ניתן לקונן (nest) אותה בתוכו. כדי לפשט את העניין זה יכול להיות מבנה (Structure). הנה מה שנקבל:

```
class String
{
public:
    ...                               // the usual String member
                                     // functions go here

private:
    struct StringValue               // nested struct
    { ... };                         // holds a reference count
                                     // and a string value

    StringValue *value;              // value of this String
};
```

הנה הפירוט של המבנה בתוך מחלקת string:

```
class String
{
private:
    struct StringValue
    {
        int refCount;
        char *data;

        StringValue(const char *initValue);
        ~StringValue();

    };

    ...

};
```

```
String::StringValue::StringValue(const char *initValue)
: refCount(1)
{
    data = new char[strlen(initValue) + 1];
    strcpy(data, initValue);
}
```

```
String::StringValue::~~StringValue()
{
    delete [] data;
}
```

מה עם העתקות והשמות?
 מה עם עדכון השדה של המספור (refCount)?
 ובכן, אלו יהיו תפקידים נוספים שנפיל על String. בינתיים, רצינו מקום עבור כל ערך עם המספור שלו בצמוד, ואת זה אכן קיבלנו מהמבנה.

נתחיל לעבור על הפונקציות של String. בנאים:

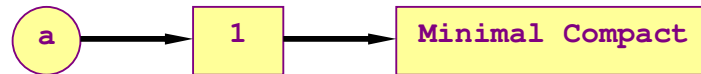
```
class String
{
public:
    String(const char *initValue = "");
    String(const String& rhs);
    ...
};
```

```
String::String(const char *initValue)
: value(new StringValue(initValue))
{}
```

קוד כזה:

```
String a("Minimal Compact");
```

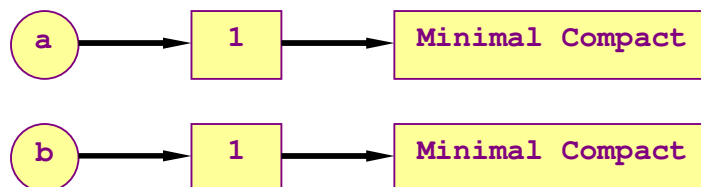
גורר יצירת מבנה כזה:



קוד כזה:

```
String a("Minimal Compact");  
String b("Minimal Compact");
```

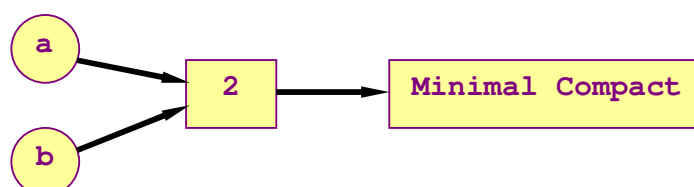
יוצר:



בהעתקות באה לידי ביטוי חלוקת הערך ע"י כמה אובייקטים:

```
String::String(const String& rhs)  
: value(rhs.value)           // make 'this' point to  
{                             // the same StringValue  
                               // as rhs  
  
    ++value->refCount;        // increase the counter  
}                              // of this StringValue
```

```
String a("Minimal Compact");  
String b = a;
```



הנה החיסכון הגדול:

✚ אין צורך להקצות מקום נוסף עבור עוד מחרוזת.

✚ אין צורך לשחרר בסוף את המקום שהוקצה.

✚ אין צורך לבצע את העתקת העומק של הערכים בין שתי מחרוזות.

במקום כל זה: העתקנו מצביע והגדלנו מונה ב-1.

הפונקציה ההורסת:

יש להקטין ב-1 את המונה באובייקט עליו מצביעים. אם הגענו ל-0, מסתבר שאובייקט String זה הוא האחרון שהצביע לשם ואז יש למחוק את ההקצאה עבור אותו StringValue.

```
class String
{
public:
    ~String();
    ...
};

String::~String()
{
    if (--value->refCount == 0) delete value;
}
```

רק נסו להשוות את היעילות של הפונקציה ההורסת פה, לזו של אובייקט String רגיל. האחרון תמיד מבצע שחרור זיכרון, בעוד שפה, מתי שיש ערכים זהים יש בדרך כלל רק הקטנת ערך ב-1 והשוואה ל-0.

פעולת השמה:

```
class String
{
public:
    String& operator=(const String& rhs);
    ...
};

s1 = s2;           // s1 and s2 are both String objects
```

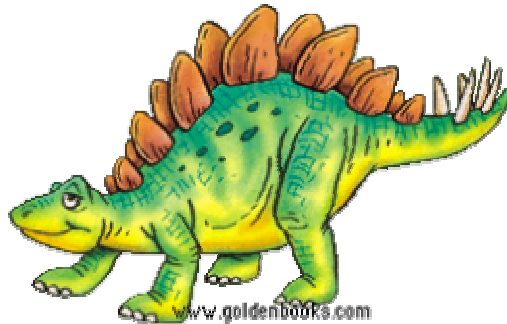
מה שאמור לקרות הוא ש-s1 ו-s2 יצביעו לאותו אובייקט StringValue. זאת אומרת שנוספה הצבעה למקום הזה, אז יש להגדיל את המונה שם. בנוסף, יש להקטין את ההצבעה באובייקט StringValue עליו s1 הצביע קודם, כי הוא לא יצביע לשם יותר (הפחתה של הצבעה). אם הוא היה האחרון שהצביע לשם, יש גם למחוק את ההקצאה ההיא.

```
String& String::operator=(const String& rhs)
{
    if (value == rhs.value)    // do nothing if the values
        return *this;         // are already the same; this
                                // subsumes the usual test of
                                // this against &rhs

    if (--value->refCount == 0) // 'leave' the old object
        delete value;         // that was pointed;

    value = rhs.value;         // have *this share
    ++value->refCount;          // rhs's value

    return *this;
}
```



העתקה עבור כתיבה (Copy-on-Write)

נחשוב על פעולת הקריאה והכתיבה עבור תא בודד במחרוזת.

```
class String
{
public:
    const char&
        operator[](int index) const;    // for const Strings

    char& operator[](int index);        // for non-const
                                        // Strings
    ...
};
```

פונקציה זו תיקרא רק עבור קריאה. הרי לא ניתן לשנות אובייקט קבוע. לכן אובייקט כזה יקרא לפונקציה זו ולעולם לא לשנייה. רצוי כמובן לבדוק לחוקיות את האינדקס אליו מנסים לגשת, פה קיצרנו.

```
const char& String::operator[](int index) const
{
    return value->data[index];
}
```

הפונקציה השנייה (non-const) עלולה להיקרא עבור קריאה וגם כתיבה (ניתן לשנות אובייקט שאינו קבוע):

```
String s;

...

cout << s[3];    // this is a read
s[5] = 'x';      // this is a write
```

כשמדובר על מספור ייחוסים, כתיבה דורשת התייחסות שונה מקריאה. אם נתייחס בצורה דומה לקריאה, מתי שיקראו לפונקציה עבור כתיבה, נשנה לא רק את האובייקט שקרא לפעולה, אלא את כל האחרים שמצביעים על אותו ערך!

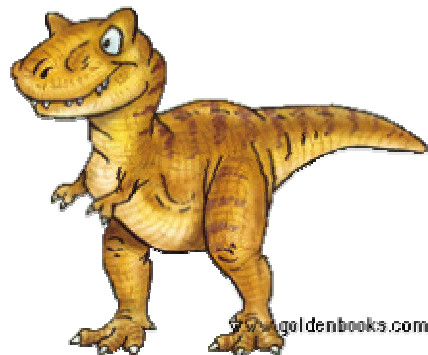
כיוון שאין אפשרות לדעת אם הפונקציה נקראה ברגע מסויים עבור קריאה או כתיבה, נתייחס באופן המחמיר לכך שזו עלולה להיות כתיבה. וכתיבה, דורשת התייחסות שונה.

אם האובייקט שקרא לפונקציה הוא היחיד שכרגע מתייחס לערך, אין בעייה. במקרה אחר, יש ליצור ערך חדש עצמאי ולהתייחס אליו מרגע זה. אז ניתן להחזיר ייחוס לאחד התאים, בלי לחשוש שנקלקל את הערך לאחד האובייקטים האחרים:

```
char& String::operator[](int index)
{
    // if we're sharing a value with other String objects,
    // break off a separate copy of the value for ourselves
    if (value->refCount > 1)
    {
        --value->refCount;           // decrement current
                                     // value's refCount,
                                     // because we won't
                                     // be using that
                                     // value any more

        value =                       // make a copy
            new StringValue(value->data); // of the value
                                     // for ourselves
    }

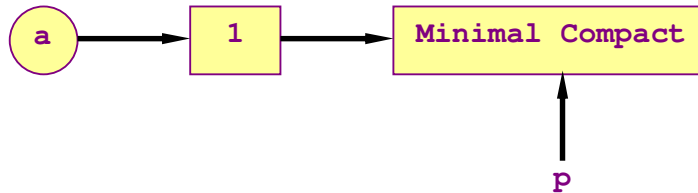
    return value->data[index];       // return a reference
}                                   // to a cell
```



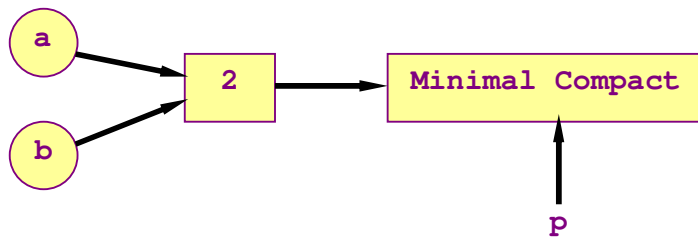
מצביעים, ייחוסים והעתקה עבור כתיבה (Copy-on-Write)

חלוקת הערך יוצרת בעיות בעוד מצבים:

```
String a = "Minimal Compact";  
char *p = &s1[1];
```



```
String b = s1;
```



בניגוד למה שניתן להבין מהקוד, כעת ניתן לשנות את שני האובייקטים דרך המצביע:

```
*p = 'x'; // modifies both s1 and s2!
```

אין כל דרך בה יכולנו לאתר את הבעייה, כיוון שאין לפונקציית ההעתקה של דרך לדעת אם לאובייקט ממנו מעתיקים מכון מצביע. אותה בעייה יש אם ישנו ייחוס אל אובייקט זה.

אין ברירה. כדי למנוע את הבעיה יש לסחוב עמנו עוד מידע.
לא המון מידע.
מספיק ביט אחד.

יש להוסיף דגל לכל אובייקט `StringValue`. דגל זה מציין אם ניתן לחלוק את האובייקט הזה.

נאתחל את הדגל ל-`true`.
ברגע שהפעילו את פעולה של כתיבה עבור אובייקט זה, נשנה ל-`false`.
דגל שהשתנה, לא יחזור לעולם להיות `true`.

```
class String
{
private:
    struct StringValue
    {
        int refCount;
        bool shareable;
        char *data;
        StringValue(const char *initValue);
        ~StringValue();
    };
    ...
};

String::StringValue::StringValue(const char *initValue)
:    refCount(1),
    shareable(true)           // add this
{
    data = new char[strlen(initValue) + 1];
    strcpy(data, initValue);
}

String::StringValue::~~StringValue()
{
    delete [] data;
}
```

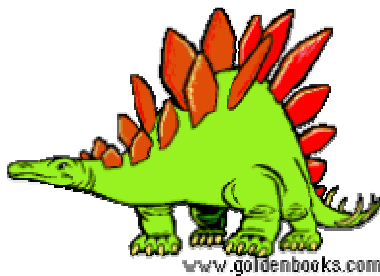
יש לעדכן גם את מחלקת String לשינוי:

```
String::String(const String& rhs)
{
    if (rhs.value->shareable)           // there is no one
    {                                   // pointing at or
        value = rhs.value;              // refering to rhs
        ++value->refCount;
    }
    else                                 // rsh is
    {                                   // shared
        value =
            new StringValue(rhs.value->data);
    }
}
```

```
char& String::operator[](int index)
{
    if (value->refCount > 1)
    {
        --value->refCount;
        value = new StringValue(value->data);
    }

    value->shareable = false;           // worn the others
                                         // that I might be
                                         // shared

    return value->data[index];
}
```



מחלקת בסיס עבור Reference Counting

מספור ייחוסים היא שיטה כללית, שנחמד היה לו יכולנו בקלות יחסית להוסיף אותה לכל מחלקה. לא רק באופן פרטני ל-String למשל. יש ליצור מעין מודול אותו נוכל להוסיף מודולרית מתי שנצטרך.

צעד 1:

יש ליצור מחלקת בסיס עבור Reference Counting. כל מחלקה, שתצצה להוסיף את היכולת להשתמש בשיטה זו, תצטרך לרשת ממנה. מחלקה זו מתחזקת את המספור עצמו וכן את הדגל האומר אם ניתן לחלוק את הערך.

```
class RObject
{
public:
    RObject();
    virtual ~RObject() = 0;
    RObject(const RObject& rhs);
    RObject& operator=(const RObject& rhs);
    void addReference();
    void removeReference();
    void markUnshareable();
    bool isShareable() const;
    bool isShared() const;

private:
    int refCount;
    bool shareable;
};
```

מה יכולה מחלקה זו לעשות?

- להיבנות ולהיהרס, אך רק כחלק ממחלקה יורשת.
- ניתן להוסיף / להחסיר לה ייחוסים.
- ניתן לבדוק לשיתופיות, ולשנות את מצב השיתופיות.

```

RObject::RObject()
: refCount(0), shareable(true) {}

RObject::RObject(const RObject&)
: refCount(0), shareable(true) {}

RObject& RObject::operator=(const RObject&)
{ return *this; }

RObject::~~RObject() {}           // virtual dtors must always
                                   // be implemented, even if
                                   // they are pure virtual
                                   // and do nothing

void RObject::addReference()
{
    ++refCount;
}

void RObject::removeReference()
{
    if (--refCount == 0) delete this;
}

void RObject::markUnshareable()
{
    shareable = false;
}

bool RObject::isShareable() const
{
    return shareable;
}

bool RObject::isShared() const
{
    return refCount > 1;
}

```

ראיתם נכון. הבנאים מאתחלים את שדה המספור ל-0. אבל לפחות מי שיצר את האובייקט מתייחס אליו! מה קורה פה?

הכוונה היא לאפשר ליוצרים של מחלקות מספור לשנות את המספור בעצמם. אם נעשה זאת פה, שם זה יראה חסר... נראה בהמשך איך עושים זאת שם ואז זה יהיה יותר ברור.

פונקציית ה- copy constructor מאתחלת תמיד את refCount ל-0. מי שייצור את האובייקט יהיה אחראי לשנות לערך המתאים.

פעולת ההשמה נראית גם מוזרה: לא נעשה כלום. למה?
זוהי מחלקת בסיס עבור אובייקטים שיחזיקו ערך. אובייקטים כאלה אינם מושמים. האובייקטים המצביעים עליהם מושמים. כמו בדוגמה שראינו: אין פעולות השמה בין אובייקטים מסוג `StringValue`. ישנן פעולות השמה בין אובייקטים של `String`. ואז מה שמשתנה ב-
`StringValue` הוא רק המספור.

עדיין, יתכן ובעתיד יבוא מי שירצה כן לאפשר השמות גם בין אובייקטים כאלה. לכן הדבר הנכון הוא כן להגדיר פעולת השמה ריקה. נניח שירצו לבצע השמה בין אובייקטים של
`StringValue`:

```
sv1 = sv2;           // how are sv1's and sv2's  
                      // reference counts affected?
```

לפני ההשמה, מספר אובייקטים התייחסו ל- `sv1`. מספר זה לא השתנה, כי מה שהשתנה הוא הערך בלבד. אותו דבר עם `sv2`, מספר האובייקטים שהתייחסו אליו לא השתנה. לכן, בפעולת השמה עבור אובייקטים כאלה אין צורך לשנות שום מספור!
וזה בדיוק מה שאופרטור ההשמה של המחלקה שראינו עושה.
כלום.

הפונקציה `removeReference` אחראית לא רק על הפחתת המספור, אלא גם על הריסת האובייקט אם המספור ירד ל-0, זאת אומרת כשאף אובייקט כבר לא מתייחס לערך. האובייקט פשוט מוחק את עצמו.

🔗🔗 צעד 2:

כדי להשתמש בפלא החדש שיצרנו, עלינו להגדיר מחדש את מחלקת `StringValue` כך שתירש ממנו. כעת מחלקה זו לא אחראית יותר על שינוי המספור. את זה מחלקת הבסיס החדשה שהגדרנו עושה במקומה:

```
class String
{
private:
    struct StringValue: public RCOBJECT
    {
        char *data;

        StringValue(const char *initValue);
        ~StringValue();
    };

    ...
};
```



```
String::StringValue::StringValue(const char *initValue)
{
    data = new char[strlen(initValue) + 1];
    strcpy(data, initValue);
}
```

```
String::StringValue::~~StringValue()
{
    delete [] data;
}
```



אוטומציה של מספור ייחוסים

המחלקה שהוספנו עבור מספור ייחוסים (RCObject) שיפרה מספר דברים:
✚ ישנו כעת מקום נפרד, בו יוחזק המונה עבור הייחוסים.
✚ ישנו מקום נפרד עבור הפונקציות, הפועלות על המונה הזה.

עדיין יש צורך לקרוא לפונקציות האלו מתוך מחלקה אחרת, כמו String. היה טוב יותר לו יכולנו לשחרר מחלקות כמו String ממחויבות כזו. אז לא יהיה צורך "ללכלך" את הקוד של מחלקות שאולי קיימות כבר בקוד של מספור ייחוסים.

שימו לב לכך שכל אובייקט String מכיל מצביע לאובייקט StringValue, המכיל את הערך ממשי:

```
class String
{
private:
    struct StringValue: public RCOBJECT { ... };

    StringValue *value;           // value of this String
    ...
};
```

אובייקט String חייב לדאוג לשינויים בשדה המספור של אובייקט StringValue עליו הוא מצביע, בכל פעם שקורה משהו שדורש שינוי: העתקה, השמה, הריסה. אם המצביע עצמו היה יכול לזהות את הפעולות האלו ולבצע את העבודה בעצמו, אובייקטים של String יהיו משוחררים מהנטל.

לשם כך יש ליצור משהו שמתנהג בדומה למצביע, אך יכול לעשות יותר. כאלה הם מצביעים חכמים.

עלינו לממש מצביע חכם, שיקח על עצמו גם את נושא המספור:

```
// template class for smart pointers-to-T objects.
// T must support the RCOBJECT interface,
// typically by inheriting from RCOBJECT
template<class T>
class RCPtr
{
public:
    RCPtr(T* realPtr = 0);
    RCPtr(const RCPtr& rhs);
    ~RCPtr();
    RCPtr& operator=(const RCPtr& rhs);
    T* operator->() const;
    T& operator*() const;

private:
    T *pointee;           // dumb pointer-to-T
    void init();          // common initialization
};                        // code
```


במצב בו יש ליצור עותק חדש, כיוון שלא ניתן לחלוק את הערך אותו קיבלנו כפרמטר, נוצר קוד כזה:

```
pointee = new T(*pointee);
```

סוג המצביע הוא T, כך שיוצרים פה אובייקט חדש מסוג T ע"י קריאה ל- copy constructor של T. במקרה שלנו T הינו StringValue. היות ולמחלקה זו אין copy constructor יתבצע כזה של ברירת מחדל. יועתק רק המצביע ולא ערך עצמאי כמו שרצינו.

נספק את הפונקציה הדרושה:

```
class String
{
private:
    struct StringValue: public RCOBJECT
    {
        StringValue(const StringValue& rhs);
        ...
    };
    ...
};

String::StringValue::StringValue(const StringValue& rhs)
{
    data = new char[strlen(rhs.data) + 1];
    strcpy(data, rhs.data);
}
```

בעצם ישנן שלוש הנחות עליהן אנו מסתמכים במימוש RCPtr<T>.

- ✚ לטיפוס T מוגדר Copy Constructor.
- ✚ הטיפוס T יורש ממחלקת RCOBJECT.
- ✚ אם קיבלנו מצביע מטיפוס T, אנו באמת מצביעים על אובייקט T. זה לא מובן מאליו, כי זה יכול להיות אובייקט של ממחלקה היורשת מ-T. במקרה שלנו נניח כי אין מי שירצה לרשת מ-StringValue. אם זה לא המקרה, יש לדעת לאיזה Copy Constructor לקרוא. אפשר לפתור זאת ע"י פונקציית בנייה וירטואלית (Virtual Copy Constructor).

מימוש שאר הפעולות של המצביע החכם:
פעולת ההשמה צריכה לבדוק אם ניתן לחלוק את הערך ממנו משימים. הפונקציה `init` עושה בדיוק את זה:

```
template<class T>
RCPtr<T>& RCPtr<T>::operator=(const RCPtr& rhs)
{
    if (pointee != rhs.pointee)           // skip assignments
    {                                       // where the value
                                           // doesn't change

        if (pointee)
        {
            pointee->removeReference();    // remove reference to
                                           // current value

            pointee = rhs.pointee;         // point to new value
            init();                         // if possible, share it
        }                                  // else make own copy

    return *this;
}
```

הפונקציה ההורסת רק צריכה לגרום להורדת המספור בערך אליו התייחסנו:

```
template<class T>
RCPtr<T>::~~RCPtr()
{
    if (pointee) pointee->removeReference();
}
```

אם מצביע זה היה האחרון להצביע לאותו הערך, הערך ישמיד את עצמו. כזכור, זו תכונה שהוא יורש ממחלקת הבסיס לאובייקטים של מספור ייחוסים:

```
void RCOBJECT::removeReference()
{
    if (--refCount == 0) delete this;
}
```

החלק המוכר של כל מצביע אוטומטי, ניתן לממש כך:

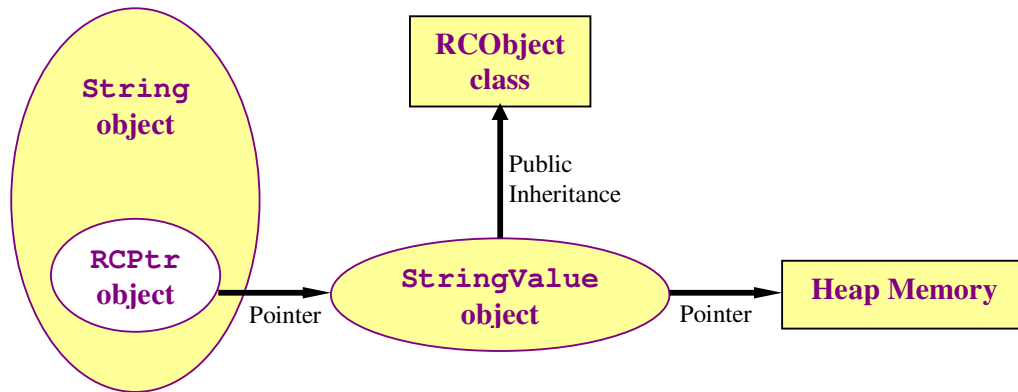
```
template<class T>
T* RCPtr<T>::operator->() const
{
    return pointee;
}
```

```
template<class T>
T& RCPtr<T>::operator*() const
{
    return *pointee;
}
```



הרכבת כל החלקים

המשטרה הייתה לבנות מחלקת String הממומשת ע"י מספור ייחוסים. נראה איך הכל עובד יחד:



```
template<class T>                // template class for smart
class RCPtr                      // pointers-to-T objects; T
{                                // must inherit from RObject
public:
    RCPtr(T* realPtr = 0);
    RCPtr(const RCPtr& rhs);
    ~RCPtr();

    RCPtr& operator=(const RCPtr& rhs);

    T* operator->() const;
    T& operator*() const;

private:
    T *pointee;

    void init();
};
```


השינוי שיחיד הוא הפונקציה `init` במחלקת `StringValue`, שתפקידה למנוע כפילויות.

לגבי מחלקת `String` יש יותר שינויים: הפונקציות שנעלמו הן:

`Copy Constructor` 
`Assignment operator` 
`Destructor` 

למה הן חסרות???

הן פשוט לא נחוצות יותר.

הן היו נחוצות כשהמחלקה הכילה **מצביע** לשטח זיכרון. כעת היא מכילה **אובייקט**. אובייקט מסוג **מצביע** חכם. כיוון שבמצביע החכם הוא זה שמכיל את המצביע ממש, הוא גם זה שדואג לשלושת הפעולות הללו, שכל מחלקה שלה מצביע לערך צריכה לדאוג להן.

מימוש מחלקת `RObject`: 

```
RObject::RObject()  
: refCount(0), shareable(true)  
{}
```

```
RObject::RObject(const RObject&)  
: refCount(0), shareable(true)  
{}
```

```
RObject& RObject::operator=(const RObject&)  
{  
    return *this;  
}
```

```
RObject::~~RObject()  
{}
```

```
void RObject::addReference()  
{  
    ++refCount;  
}
```

```
void RObject::removeReference()  
{  
    if (--refCount == 0) delete this;  
}
```

```

void RCOBJECT::markUnshareable()
{
    shareable = false;
}

bool RCOBJECT::isShareable() const
{
    return shareable;
}

bool RCOBJECT::isShared() const
{
    return refCount > 1;
}

```

:RCPtr מימוש 

```

template<class T>
void RCPtr<T>::init()
{
    if (pointee == 0) return;
    if (pointee->isShareable() == false)
    {
        pointee = new T(*pointee);
    }
    pointee->addReference();
}

```

```

template<class T>
RCPtr<T>::RCPtr(T* realPtr)
: pointee(realPtr)
{
    init();
}

```

```

template<class T>
RCPtr<T>::RCPtr(const RCPtr& rhs)
: pointee(rhs.pointee)
{
    init();
}

```

```

template<class T>
RCPtr<T>::~~RCPtr()
{
    if (pointee) pointee->removeReference();
}

template<class T>
RCPtr<T>& RCPtr<T>::operator=(const RCPtr& rhs)
{
    if (pointee != rhs.pointee)
    {
        if (pointee) pointee->removeReference();
        pointee = rhs.pointee;
        init();
    }
    return *this;
}

template<class T>
T* RCPtr<T>::operator->() const
{
    return pointee;
}

template<class T>
T& RCPtr<T>::operator*() const
{
    return *pointee;
}

```

מימוש :StringValue 

```

void String::StringValue::init(const char *initValue)
{
    data = new char[strlen(initValue) + 1];
    strcpy(data, initValue);
}

String::StringValue::StringValue(const char *initValue)
{
    init(initValue);
}

```

```
String::StringValue::StringValue(const StringValue& rhs)
{
    init(rhs.data);
}
```

```
String::StringValue::~~StringValue()
{
    delete [] data;
}
```

מימוש (מה שנשאר לממש) מחלקת String:

```
String::String(const char *initValue)
: value(new StringValue(initValue))
{}

```

```
const char& String::operator[](int index) const
{
    return value->data[index];
}
```

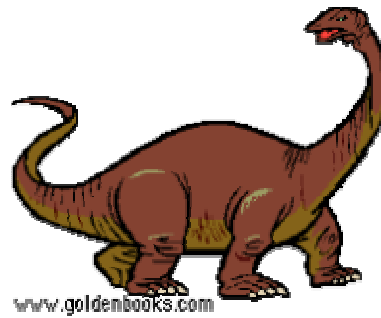
```
char& String::operator[](int index)
{
    if (value->isShared())
    {
        value = new StringValue(value->data);
    }

    value->markUnshareable();
    return value->data[index];
}
```

ההבדלים העיקריים ממחלקת String במימוש הראשון:

יש פחות מה לממש – חלקים שעברו אל המצביע החכם. כעת אנו מכילים אובייקט ולא מצביע. אובייקט יכול לעשות בעצמו את המניפולציות על עצמו.
 הקוד של הפעולות שנשארו כמעט ולא השתנה. מה שכן השתנה נמצא ב-
 String::operator[] שם המניפולציות על הערך ושדה המספור של StringValue
 נעשות דרך פונקציות.

😊 . קוד שמשתמש במחלקת String לא צריך לעבור שום שינויים כדי שיתאים לשיטה החדשה! הדבר היחיד שיש לבצע עבור קוד קיים הוא קומפילציה ובנייה מחדש.



הוספת מספור ייחוסים למחלקות קיימות

כל מה שעשינו עד עכשיו התבסס על כך שיש לנו גישה לקוד של המחלקה, שעבורה אנו רוצים לממש מספור ייחוסים.

ואם אין לנו גישה?

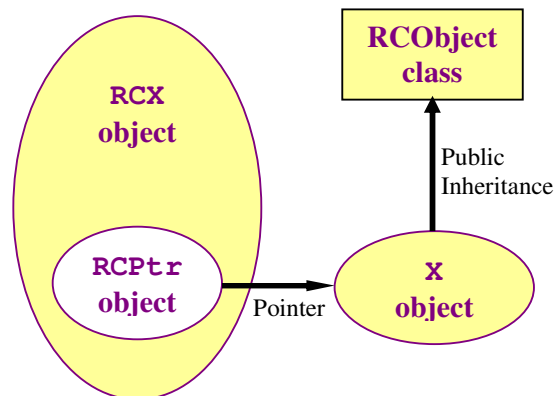
אם מדובר באיזושהי מחלקת X שניתן לקבל רק הרשאות קריאה אליה?

אם אין לנו אפשרות לגרום למחלקה זו לרשת ממחלקת RCOBJect, אז לא ניתן להשתמש במצביעים חכמים כמו שעשינו קודם.

איך לעשות כך שניתן יהיה להוסיף מספור ייחוסים לכל מחלקה?
צריך לשנות מעט את העיצוב, קודם כל.

👉 צעד 1:

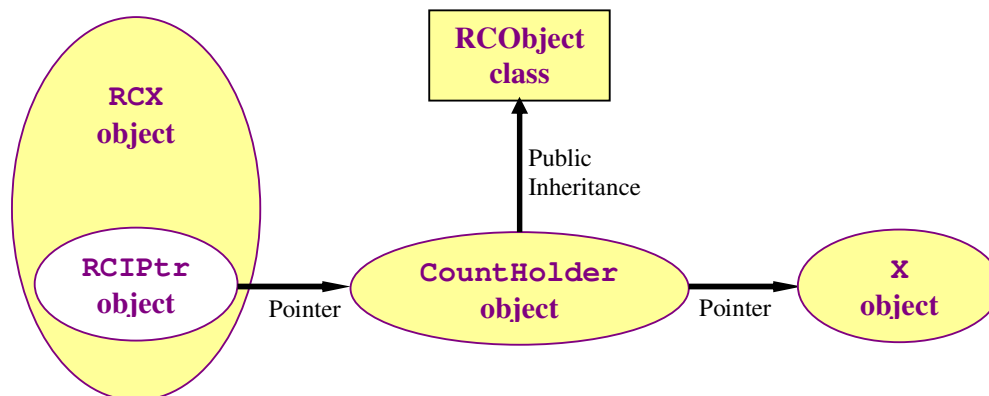
נניח כי מחלקת X יכולה לרשת ממחלקת RCOBJect. במקרה כזה יהיה עלינו להוסיף מחלקה בשם RCX, בה ישתמשו. הכל עדיין יהיה אנלוגי לדוגמה שכבר עשינו, כאשר RCX תשחק את התפקיד של String ו-X את התפקיד של StringValue.



👉👉 צעד 2:

נוסיף מחלקה חדשה, CountHolder, שהיא זו שתכיל את מספור הייחוסים. מחלקה זו תירש ממחלקת RCOBJect. אז נחליף את המצביע ל-RCIPtr (Reference Count Indirect) שיודע על קיומה של המחלקה CountHolder.

זה מה שנקבל:



באותו אופן שמשתמשים של String לא הכירו את StringValue, כך משתמשים של RCX לא יכירו את CountHolder. בכלל, הוא יהיה חבוי (ההגדרה שלו) בתוך RCIPtr.

```

template<class T>
class RCIPtr
{
public:
    RCIPtr(T* realPtr = 0);
    RCIPtr(const RCIPtr& rhs);
    ~RCIPtr();

    RCIPtr& operator=(const RCIPtr& rhs);

    const T* operator->() const;
    T* operator->();
    const T& operator*() const;
    T& operator*();

private:
    struct CountHolder: public RObject          // the nested
    {                                           // CountHolder
        ~CountHolder() { delete pointee; }
        T *pointee;
    };

    CountHolder *counter;

    void init();
    void makeCopy();                          // see below
};
  
```

```

template<class T>
void RCIPtr<T>::init()
{
    if (counter->isShareable() == false)
    {
        T *oldValue = counter->pointee;
        counter = new CountHolder;
        counter->pointee = new T(*oldValue);
    }

    counter->addReference();
}

```

```

template<class T>
RCIPtr<T>::RCIPtr(T* realPtr)
: counter(new CountHolder)
{
    counter->pointee = realPtr;
    init();
}

```

```

template<class T>
RCIPtr<T>::RCIPtr(const RCIPtr& rhs)
: counter(rhs.counter)
{
    init();
}

```

```

template<class T>
RCIPtr<T>::~~RCIPtr()
{
    counter->removeReference();
}

```

```

template<class T>
RCIPtr<T>& RCIPtr<T>::operator=(const RCIPtr& rhs)
{
    if (counter != rhs.counter)
    {
        counter->removeReference();
        counter = rhs.counter;
        init();
    }
    return *this;
}

```



```

template<class T>                                // implement the copy
void RCIPtr<T>::makeCopy()                        // part of copy-on-
{                                                  // write (COW)
    if (counter->isShared())
    {
        T *oldValue = counter->pointee;
        counter->removeReference();
        counter = new CountHolder;
        counter->pointee = new T(*oldValue);
        counter->addReference();
    }
}

template<class T>                                // const access;
const T* RCIPtr<T>::operator->() const           // no COW needed
{
    return counter->pointee;
}

template<class T>                                // non-const
T* RCIPtr<T>::operator->()                        // access; COW
{                                                  // needed

    makeCopy(); return counter->pointee;
}

template<class T>                                // const access;
const T& RCIPtr<T>::operator*() const           // no COW needed
{
    return *(counter->pointee);
}

template<class T>                                // non-const
T& RCIPtr<T>::operator*()                       // access; COW
{                                                  // needed

    makeCopy(); return *(counter->pointee);
}

```

יש שני הבדלים בין RCIptr ל-RCPtr:
 ✚ RCIptr הצביע לערכים ישירות, בעוד ש-RCPtr מערב אובייקט של CountHolder.
 ✚ RCIptr מגדיר את האופקטורים '>', '*', ' * ' בצורה כזו, שפעולת Copy-on-Write מתבצעת באופן אוטומטי כשצריך.

כעת אין בעיה לממש את RCX.
 נניח שמחלקת X נראית כך:

```
class X
{
public:
    X(int size);
    X(const X& rhs);
    ~X();
    X& operator=(const X& rhs);
    void doThis();
    int showThat() const;
};
```

כך תיראה RCX:

```
class RCX
{
public:
    RCX(int size): value(new X(size)) {}
    void doThis() { value->doThis(); }
    int showThat() const { return value->showThat(); }

private:
    RCIptr<X> value;
};
```

קלי קלות.
 כמו בפתרון הראשון, גם פה אין צורך בשלישיה
 ✚ Copy Constructor
 ✚ Assignment Operator
 ✚ Destructor

זה לא מקרה, אלא בדיוק מאותה סיבה: המחלקה מכילה לא מצביע, אלא אובייקט (מצביע חכם).
 הוא דואג לדברים כאלה, בהיותו אובייקט (וחכם).



מילות סיום

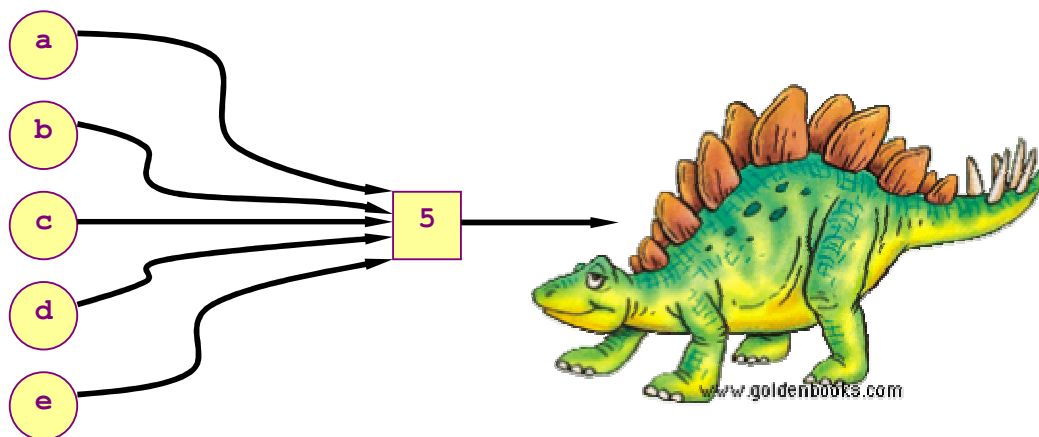
מתי כדאי להשתמש בשיטה של מספור ייחוסים?

אם נשתמש בשיטה זו -

✚ ככל שאובייקטים מכילים ערכים גדולים יותר, והרבה אובייקטים יחזיקו את אותו ערך, נחסוך מקום בזיכרון.

✚ ככל שירבו פעולות השמה והעתקה בין אובייקטים, נחסוך זמן.

✚ ככל שלוקח זמן רב יותר לבנות ולהרוס אובייקטים, נחסוך זמן. זאת בהנחה שרבים מהם יחזיקו את אותו ערך.



צריך לזכור, כי אם התנאים לא מתקיימים, אנחנו מורידים מאיכות הביצועים של התכנית:

⊗ כל ערך סוחב איתו מספור (עלות מקום).

⊗ יש לבצע מניפולציות (בדיקות ושינויים) על המספור הנ"ל (עלות זמן).